Once you understand that, we can take a look at the major players and where they sit. There isn't a single right answer in the end, but you'll have a solid grasp of why we pick Req as our preferred client.

The Various Levels of Abstraction

Let's walk through the different levels of abstraction from the lowest to the highest.

Sockets

At the lowest level, HTTP requests are made over sockets. These sockets usually speak TCP.

It's not common to interface with a raw socket. Erlang provides the `:gen_tcp` module to interact with raw sockets over TCP.

Socket Protocol

Sockets exchange packets back and forth, but they need a defined protocol to communicate effectively with each other. In an HTTP API, the HTTP/1 or HTTP/2 protocol will be used.

Socket protocols are usually implemented by libraries that serve as a foundation for other libraries. For example, the Postgrex[1] library implements the Postgres protocol on top of a TCP socket.

Pooled Connections

A socket is a single connection, but we need multiple, simultaneous connections to scale our application. Pooled connections use a socket protocol library and add connection pooling to it.

It's important to have a handle on connection pooling as you scale an application. If your pool is too small, then you won't be able to achieve the scalability you need. If your pool is too large, then you can send too many requests to another service and cause it to crash.

A non-HTTP example of this is the pooling library PgBouncer. This library only pools Postgres connections, but it's one of the most important tools for scaling Postgres to high throughput.

All-Included Libraries

At the highest level of abstraction, there are libraries that provide all of the bells and whistles you'll need when building your own application.

Simple libraries will provide minimal features but generally give you what you need to be successful. Extensible libraries give you options that you

1. https://github.com/elixir-ecto/postgrex